

During fine-tuning, we optimize the matrices $\mathbf{W}_q^i, \mathbf{W}_k^i, \mathbf{W}_v^i$ in the attention layer and $\mathbf{W}_1, \mathbf{W}_2, \mathbf{b}_1, \mathbf{b}_2$ in the MLP layer pertaining to a loss function $\mathcal{L}$. However in prompt tuning, the pretrained model weight matrices are fixed and we optimize a tunable sequence prepended to the input.

Prompt Tuning Given a pretrained transformer network $g \in \mathcal{T}$ and a downstream training dataset $S = \{(\mathbf{X}_1, \mathbf{Y}_1), \dots, (\mathbf{X}_n, \mathbf{Y}_n)\}$, prompt tuning seeks to find a prompt $\mathbf{P}^* \in \mathbb{R}^{d \times m_p}$ with m_p tunable tokens under the loss function $\mathcal{L}$:

$$\mathbf{P}^* = \arg \min_{\mathbf{P}} \sum_{i=1}^n \mathcal{L}(g([\mathbf{P}, \mathbf{X}_i])_{:,m_p:}, \mathbf{Y}_i). \quad (4)$$

The tunable prompt $\mathbf{P}$ is shared amongst all the inputs in a task. Note that $\mathbf{P}$ in prompt tuning is a continuously trainable parameter, alternately referred to as soft prompt, which is different from hard prompt in that the latter operates on a discrete space of predefined vocabulary. Since the representation power of soft prompts is strictly more than the hard prompts, the limitations studied in this paper also extend to hard prompts.

In the subsequent sections, we analyze the universality and limitations of prompt tuning while comparing the latter against fine-tuning and LoRA [Hu et al., 2021], which is a low-rank version of model fine-tuning. In Section 4, we prove that prompt tuning can be universal approximators for sequence-to-sequence functions, while providing the construction for the same. In Sections 5 and 6, we identify the failure modes where prompt tuning cannot learn with a possibly non-optimal but non-trivial pretrained transformer network.

4 Universality of Prompt Tuning

Without loss of generality, we assume that the support and range set of all considered sequence-to-sequence functions f is $[0, 1]^{d \times m}$ in this section. We define $\mathcal{F}_L$ as the collection of all continuous sequence-to-sequence L -lipschitz functions under norm p and sequence length m . For $f \in \mathcal{F}_L$ and any two inputs $\mathbf{X}, \mathbf{X}' \in [0, 1]^{d \times m}$, we have $\|f(\mathbf{X}) - f(\mathbf{X}')\|_p \leq L \|\mathbf{X} - \mathbf{X}'\|_p$. Furthermore, given functions f_1, f_2 , the approximation error under a p -norm (which is entry-wise) is measured as:

$$d_p(f_1, f_2) = \left(\int \|f_1(\mathbf{X}) - f_2(\mathbf{X})\|_p^p d\mathbf{X} \right)^{\frac{1}{p}}. \quad (5)$$

Primarily, we show that there exists a Transformer network $g \in \mathcal{T}^{2,1,4}$ such that for any $f \in \mathcal{F}_L$, prompt tuning on g can approximate this function upto some error budget $\epsilon > 0$.

Theorem 1. *Let $1 \leq p < \infty$ and $\epsilon > 0$, there exist a transformer network $g \in \mathcal{T}^{2,1,4}$ and prompt length m_p , such that for any $f \in \mathcal{F}_L$ we can find a prompt $\mathbf{P} \in \mathbb{R}^{d \times m_p}$ with $d_p(g([\mathbf{P}, \cdot])_{:,m_p:}, f) \leq \epsilon$.*

Here we use the transformer in a encoder mode which generates the m outputs in one step. In Appendix C.4, a similar result can be obtained for next-token prediction, which is widely used in many recent language models.

The proof is inspired from [Yun et al., 2019a], which follows the typical construction based proof mechanism to show universality. Thereby, we can construct a “meta-transformer” for prompt tuning to approximate any sequence-to-sequence function with prompt tuning. Next we briefly describe the two steps for the construction of this meta-transformer. We start by building a meta-function for $\mathcal{F}_L$.

Building the Meta-Function We denote the length of all inputs as m and the prompt length as m_p . Then we can build a sequence-to-sequence meta-function that accepts inputs with length $m + m_p$.

Lemma 1. *For the sequence-to-sequence function space $\mathcal{F}_L$ with functions $f : [0, 1]^{d \times m} \rightarrow [0, 1]^{d \times m}$, we can build a sequence-to-sequence function $\bar{g} : [0, 1]^{d \times (m_p + m)} \rightarrow [0, 1]^{d \times (m_p + m)}$ such that for any $f \in \mathcal{F}_L$, we can find $\mathbf{P} \in \mathbb{R}^{d \times m_p}$, $d_p(\bar{g}([\mathbf{P}, \cdot])_{:,m_p:}, f) \leq \epsilon/2$.*

The complete proof is given in Appendix C.1. Succinctly, we first quantize the input and output sequence space of $[0, 1]^{d \times m}$ into a grid $G_{\delta, m} = \{0, \delta, 2\delta, \dots, 1 - \delta\}^{d \times m}$, thus leading to $C = (\frac{1}{\delta^{d \times m}})^{\frac{1}{\delta^{d \times m}}}$ possible functions mappings from the input to the output, in this discrete space. By

this quantized function space as $\bar{\mathcal{F}}_L = \{\bar{f}_1, \bar{f}_2, \dots, \bar{f}_C\}$, we can select δ such that the approximation error for any function is less than $\epsilon/2$. Then we construct a set of quantized prompts in $G_{\delta, m_p} = \{0, \delta, 2\delta, \dots, 1 - \delta\}^{d \times m_p}$ to index these C functions and construct a quantized function $\bar{g}$ where $\bar{g}([\mathbf{P}_i, \mathbf{X}])_{:, m_p} = \bar{f}_i(\mathbf{X})$, $i = 1, 2, \dots, C$, for all $\mathbf{X} \in G_{\delta, m}$, thereby concluding the lemma.

Next we can utilize some conclusions in [Yun et al., 2019a] to construct a transformer for $\bar{g}$.

Constructing the Meta-Transformer We first introduce a useful lemma which enables the construction of a transformer for any quantized sequence-to-sequence function.

Lemma 2. *For any given quantized function $\bar{f} : [0, 1]^{d \times m} \rightarrow [0, 1]^{d \times m}$ with quantization at interval*

$$\delta, \exists \bar{h} \in \bar{\mathcal{T}}^{2,1,1} \text{ such that } \bar{f} = \bar{h} \text{ with positional embedding } \mathbf{E} = \begin{bmatrix} 0 & 1 & 2 & \dots & m-1 \\ 0 & 1 & 2 & \dots & m-1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 1 & 2 & \dots & m-1 \end{bmatrix}.$$

The proof mainly follows the discussions in Section C of [Yun et al., 2019a]. To prove this lemma, the network $\bar{h}$ can be constructed in the following three steps. We first use a series of MLP layers to quantize the input to grid $[0 : \delta : 1 - \delta]^{d \times m}$ and then a series of attention layers to obtain a unique contextual mapping for each quantized input. Finally we can use a series of MLP layers to map the unique contextual mapping to the desired outputs. While a transformer network usually stacks self-attention and MLP layers alternately within a single layer, the aforementioned construction can be trivially attained via the use of skip connections. The complete proof of Lemma 2 is deferred to Appendix C.2.

Since $\bar{g}$ is a quantized function in grid $G_{\delta, m+m_p}$, following Lemma 2 we can find a modified version of transformer $\bar{h} \in \bar{\mathcal{T}}^{2,1,1}$ such that $\bar{g}([\mathbf{P}, \mathbf{X}]) = \bar{h}([\mathbf{P}, \mathbf{X}])$. The modified version of transformer $\bar{g}$ with hardmax operators can then be approximated with a standard transformer g with softmax operators by Lemma 3.

Lemma 3 (Lemma 9 in [Yun et al., 2019a]). *For each $\bar{h} \in \bar{\mathcal{T}}^{2,1,1}$, $\epsilon > 0$ and $1 \leq p < \infty$, $\exists g \in \mathcal{T}^{2,1,4}$ such that $d_p(\bar{h}, g) \leq \epsilon/2$.*

Since the approximation error can be treated uniformly amongst the $\mathbf{P}_i$, we have that $d_p(\bar{h}([\mathbf{P}_i, \cdot])_{:, m_p}, g([\mathbf{P}_i, \cdot])_{:, m_p}) \leq d_p(\bar{h}([\mathbf{P}_i, \cdot]), g([\mathbf{P}_i, \cdot])) \leq \epsilon/2$. Therefore, we can build a transformer $g \in \mathcal{T}^{2,1,4}$, such that for any sequence-to-sequence $f \in \mathcal{F}_L$, we can find a quantized version $\bar{f}_i \in \bar{\mathcal{F}}_L$ and the corresponding prompt $\mathbf{P}_i \in G_{\delta, m_p}$ such that

$$d_p(g([\mathbf{P}_i, \cdot])_{:, m_p}, f) \leq d_p(g([\mathbf{P}_i, \cdot])_{:, m_p}, \bar{h}([\mathbf{P}_i, \cdot])) + d_p(\bar{h}([\mathbf{P}_i, \cdot])_{:, m_p}, \bar{f}_i) + d_p(\bar{f}_i, f) \leq \epsilon. \quad (6)$$

Theorem 1 provides the construction for a large transformer (discussed more in appendix) that is sufficient for prompt tuning to exhibit universal approximation over a Lipschitz function space. However, even this strong transformer also has limitations with prompt tuning when the target function $f \notin \mathcal{F}_L$. Is this an essential limitation for prompt tuning on any transformer? In the next section, we will theoretically analyze the limitations of prompt tuning with transformers and target functions under more general conditions.

5 Limitations of Prompt-Tuning: Single Layer Transformer

To analyse the failure modes and therefore the limitations under the setting where a transformer has fixed pretrained weights, we follow the lens of exact memorization in the subsequent sections.

Definition 3 (Memorization of a Sequence-to-Sequence Dataset). *Given a sequence-to-sequence dataset $S = \{(\mathbf{X}_1, \mathbf{Y}_1), \dots, (\mathbf{X}_n, \mathbf{Y}_n)\}$ where $\mathbf{X}_i, \mathbf{Y}_i \in \mathbb{R}^{d \times m}$ are the input/output sequences, we consider a function f exactly memorizing dataset S if $f(\mathbf{X}_i) = \mathbf{Y}_i$. In the following proofs of this section, we explicitly focus on the last output token, ie: $f(\mathbf{X}_i)_{:, -1} = (\mathbf{Y}_i)_{:, -1}$.*

We start from the analysis on a single layer transformer and extend to multi-layer settings in Section 6.

5.1 Failure modes of Prompt Tuning

It is straightforward to note that prompt tuning has limited expressive power when the number of trainable parameters is limited. A natural question to then ask is: Does increasing the number of trainable prompt tokens suffice? While it is known that for MLPs, even with a single hidden layer, increasing the number of hidden neurons can memorize any training data [Yun et al., 2019b]. However, as we will prove next, this is not the case for prompt tuning. This result highlights an essential limitation of prompt tuning compared to model fine-tuning.

Before providing the theorem statement, we first outline some straightforward assumptions on the pretrained transformer and datasets, without which prompt tuning trivially loses expressive power.

We consider sequence-to-sequence datasets of the form $S = \{(\mathbf{X}_1, \mathbf{Y}_1), (\mathbf{X}_2, \mathbf{Y}_2), \dots, (\mathbf{X}_n, \mathbf{Y}_n)\}$ with n distinct examples and a single-layer single-head standard transformer defined in Definition 2. The results can be directly extended to the single-layer multi-head scenario, which we skip here to avoid notational clutter.

Assumption 1 (Non-trivial conditions). *We assume that all output tokens $(\mathbf{Y}_i)_{:,k}$ are in the range set of MLP, otherwise the expressivity becomes trivially weak. We assume that $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$ are full rank matrices and that $\text{Att}(\mathbf{X}_i, \mathbf{X}_i) + \mathbf{x}_i$ are distinct for $i = 1, 2, \dots, n$.*

Assumption 2 (Assumption for the MLP layer). *We assume that $d \geq 2 + \dim((\text{MLP}^{-1}(\mathbf{y}_{10}) - \mathbf{x}_0) \cup (\text{MLP}^{-1}(\mathbf{y}_{20}) - \mathbf{x}_0))$ for the dataset constructed in Theorem 2 and token dimension d . $\dim(S)$ measures the dimension of subspace spanned by vectors in a set S and $\text{MLP}^{-1}(\mathbf{y}) = \{\mathbf{x} : \text{MLP}(\mathbf{x}) = \mathbf{y}\}$.*

We provide an example for this assumption in Example 1 and a sufficient condition in the following Lemma 4.

Lemma 4. *If $\|\mathbf{W}_1\|_2 \times \|\mathbf{W}_2\|_2 < 1$, where $\|\cdot\|_2$ is the matrix spectral norm, then the MLP block in Definition 2 is invertible, ie, MLP^{-1} is a singleton set.*

Therefore, if Lemma 4 holds and $d \geq 4$, Assumption 2 also holds.

Proof of Lemma 4 can be found in Appendix C.5. The experimental evidence in [Dong et al., 2021] shows that for most architectures, the norm of the weight matrices indeed admits small values and thus the requirement that $\|\mathbf{W}_1\|_2 \times \|\mathbf{W}_2\|_2 < 1$ is a mild condition.

With these assumptions, here we introduce our first theorem on the unlearnability of prompt tuning.

Theorem 2. *For a single layer transformer τ defined above with Assumptions 1 and 2, we can build a sequence-to-sequence dataset $S = \{(\mathbf{X}_1 = [\mathbf{x}_1, \mathbf{x}_0], \mathbf{Y}_1 = [\mathbf{y}_{11}, \mathbf{y}_{10}]), (\mathbf{X}_2 = [\mathbf{x}_2, \mathbf{x}_0], \mathbf{Y}_2 = [\mathbf{y}_{21}, \mathbf{y}_{20}])\}$, and we cannot find a prompt $\mathbf{P} \in \mathbb{R}^{d \times m_p}$ with any $m_p > 0$ such that $\tau([\mathbf{P}, \mathbf{X}_i]) = \mathbf{Y}_i$ holds for any $i = 1, 2$. The vectors $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2$ are denoted post positional encodings.*

An important feature of this dataset is that the same token $\mathbf{x}_0$ is shared between the two examples, and the expressive capability of prompt tuning is limited by the correlation of outputs corresponding to this token in different examples. We show a concrete example here to illustrate this theorem (note that Lemma 4 is in fact not required in the following construction) and defer the formal proof to Appendix C.6.

Example 1. *We consider a single-head transformer layer τ , where $\mathbf{b}_1 = \mathbf{b}_2 = \mathbf{0}$, $\mathbf{W}_1 = \mathbf{1}^{r \times d}$, $\mathbf{W}_2 = \mathbf{1}^{d \times r}$. Then the token-wise MLP layer is a concatenation of two linear functions:*

$$\text{MLP}(\mathbf{x}) = \begin{cases} (\mathbf{W}_2 \mathbf{W}_1 + \mathbf{I})\mathbf{x}, & (\mathbf{W}_1 \mathbf{x})_0 > 0 \\ \mathbf{x}, & (\mathbf{W}_1 \mathbf{x})_0 \leq 0 \end{cases} \quad (7)$$

Here $(\mathbf{W}_1 \mathbf{x})_0$ denotes the first element of vector $\mathbf{W}_1 \mathbf{x}$.

$\mathbf{W}_2 \mathbf{W}_1 + \mathbf{I}$ is a non-singular matrix. Therefore, for any $\mathbf{y}$ in $\text{MLP}(\mathbf{X})$'s output set, $\text{MLP}^{-1}(\mathbf{y})$ contains at most two points $\{\mathbf{y}, (\mathbf{W}_2 \mathbf{W}_1 + \mathbf{I})^{-1} \mathbf{y}\}$. We arbitrarily choose $\mathbf{x}_0, \mathbf{y}_{10}$ and $\mathbf{y}_{20}$.

As long as $d \geq 6$ (from Assumption 2), we can find $\mathbf{c}_1, \mathbf{c}_2$ such that $\mathbf{c}_1, \mathbf{c}_2 \perp \mathbf{y}_{10} - \mathbf{x}_0, \mathbf{y}_{20} - \mathbf{x}_0, (\mathbf{W}_2 \mathbf{W}_1 + \mathbf{I})^{-1} \mathbf{y}_{10} - \mathbf{x}_0, (\mathbf{W}_2 \mathbf{W}_1 + \mathbf{I})^{-1} \mathbf{y}_{20} - \mathbf{x}_0, \mathbf{c}_1 \perp \mathbf{c}_2$. Then we choose $\mathbf{x}_1$ and $\mathbf{x}_2$ such that $\text{Att}(\mathbf{x}_0, \mathbf{X}_1) \parallel \mathbf{c}_1$ and $\text{Att}(\mathbf{x}_0, \mathbf{X}_2) \parallel \mathbf{c}_2$ (Lemma 7 in Appendix). Then $\text{Cone}(-\text{Att}(\mathbf{x}_0, \mathbf{X}_1), \mathbf{a} - \mathbf{x}_0) \cap \text{Cone}(-\text{Att}(\mathbf{x}_0, \mathbf{X}_2), \mathbf{b} - \mathbf{x}_0) = \emptyset$, for any $\mathbf{a} \in \{\mathbf{y}_{10}, (\mathbf{W}_2 \mathbf{W}_1 + \mathbf{I})^{-1} \mathbf{y}_{10}\}$ and $\mathbf{b} \in \{\mathbf{y}_{20}, (\mathbf{W}_2 \mathbf{W}_1 + \mathbf{I})^{-1} \mathbf{y}_{20}\}$. Here Cone stands for a convex cone as defined in Section 3.1.